

Ascon Architecture Anarchy

Mia Diliberti-
Brandenburg
miad@cpp.edu

John Pollak
jpollak@cpp.edu

Kelly Williams
kwilliams1@cpp.edu

Marcelo P. Masilungan,
Jr.
mpmasilungan@cpp.edu

Garima Bhattarai
gbhattarai@cpp.edu

Cal Poly Pomona
College of Engineering
Pomona, CA, USA

Abstract—The Ascon family of cryptographic algorithms is intended to replace and/or augment heavier authenticated key and hashing algorithms in embedded systems. By benchmarking four architectures on five machines and taking a survey of existing research, we can show how the efficiency of Ascon is affected by these architectures.

Keywords—Ascon, lightweight cryptography, AEAD, benchmarking, RISC-V, Raspberry Pi 5, x86-64

I. INTRODUCTION AND BACKGROUND

A. What is Ascon?

Ascon is a cipher suite supporting authenticated encryption (AEAD) and hashing. [1] The Internet Engineering Task Force (IETF) describes an AEAD as having specific traits, in accordance with RFC 5116. This is described as “deterministic” and with four inputs/outputs: a secret (fixed length) key, a non-repeated nonce, associated data, and either a plaintext message or ciphertext, depending on the direction of the algorithm. [2] AEADs have a strict set of guidelines that make them have data integrity and confidentiality due to their combination of authentication and strong encryption. [2] Interestingly, as well, they are intended to have a certain level of robustness against quantum attack, and are parallelizable as well as streamable, although block based AEADs are mentioned and acceptable in the IETF Internet-Draft. [2]

Ascon was proposed in the first CAESAR competition at NIST and subsequently competed in their lightweight cryptography competition. [1] The family consists of Ascon-128AEAD and 256AEAD, which are authenticated ciphers, Ascon-Hash256, a hash function, and Ascon-XOF128 and Ascon-CXOF128, extendable output functions. [1][3] Its permutation functions are similar to SHA-2, a hashing algorithm, and the encryption is based on duplex modes, whereas hashing is based on sponge construction. [1] Ascon has strong defense against side-channel attacks and is noted to be especially suitable for lightweight devices to communicate with high end servers, short message sizes, and security on constrained devices, due to its low software and hardware overhead. [1]

B. The Case For Lightweight Cryptography

TLS is an important cryptographic protocol for securing communications over the Internet and is normally implemented at the transport layer, such as on TCP, to secure application layer protocols such as HTTP and FTP. [4] At present, TLS can use one of many different protocols for key generation and exchange, such as RSA, and a few different permutations of

Elliptic Curve with Diffie-Hellman, such as ECDH. [4] The key exchange will create a session key which is discarded at the end of a TLS session. [4] TLS-1.3 discards RSA entirely for Elliptic Curve based exchange schemes, due to security concerns, and, as of May 2024, TLS-1.3 was used in 70.1% of websites that were surveyed by SSL-Lab. [5] As you download or read this document, it is very likely that you are receiving it over a session encrypted by TLS.

As of writing this paper in December 2025, Google reports that 95% of Windows traffic, 98% of Mac traffic, and a whopping 99% of Android and ChromeOS traffic through their services was over HTTPS. [6] Linux sits much lower at 88% [6] and although one can speculate why this is, it is possible that Linux runs on lower performance or more outdated embedded devices due to the variety of types of Linux that exist. The Eclipse Foundation, a large open source software foundation, reports in its 2024 survey of IoT and embedded developers that 44-46% of embedded devices run Linux. [7] This survey also reveals that many developers have connectivity and security as top concerns in the implementation and design of software in IoT and other embedded devices. [7]

This need for connectivity and security, as well as high usage of TLS connectivity, requires embedded devices use encryption via TLS when connecting over certain services.

Although the use case for Ascon is more aligned with AES, a block cipher, [8] the proliferation of TLS is a clear indication of heavy use of cryptographic algorithms in IoT devices, and provides a clear incentive for creating secure, fast, and power-saving algorithms for use in communications on embedded devices. A NIST workshop paper also points out that AES is restricted to devices with at least 512 bytes of memory and that TLS often puts stress onto embedded devices. [9] This provides justification for the existence of algorithms which can reduce overhead on embedded devices so that heavier algorithms in necessary software can run when needed.

NIST reports that AEAD algorithms are critical in applications ranging from file transfers to protecting RFID tag communications in warehouses. [8] Bluetooth (BLE) link layer security, uses the AEAD algorithm AES-128 in conjunction with the CCM message authentication code [10] and Wi-Fi, starting in WPA2, uses AES as an option for authentication; WPA3 also requires hashing with SHA-256. [11][12] Ascon-Hash256 and Ascon-128/256AEAD are potential substitutes that could be used in resource limited systems: NIST specifies that existing algorithms like AES are not intended to be

TABLE I-1: HARDWARE PLATFORMS USED

Platform	Processor	Architecture	Memory
Raspberry Pi 5	Broadcom BCM2712, quad-core ARM Cortex-A76 @ 2.4 GHz	ARMv8.2-A with FP16, RCPC, DotProd, and Crypto extensions	8 GB LPDDR4X-4267
Raspberry Pi Zero W	Broadcom BCM2835, single-core ARM1176JZF-S @ 1 GHz	ARMv6 with VFP floating-point	512 MB LPDDR2
StarFive VisionFive2	StarFive JH7110, quad-core SiFive U74 @ 1.5 GHz	RV64GC (RV64IMAFDC)	8 GB LPDDR4
Qotom Mini-PC	Intel Bay Trail Celeron SoC @ 1.83 GHz to 2.66 GHz	x86-64	Up to 8 GB DDR3L
Intel Skylake-D	Intel Xeon D-2146NT SoC @ 2.3 GHz to 3.0 GHz	x86-64	Up to 512 GB ECC DDR4-2133 LRDIMM

replaced, particularly for interoperability reasons, but that algorithms like Ascon can be used to help close the performance/security gap in resource restricted systems. [8]

J. Avery et al in their paper for the NIST lightweight workshop experimented with replacing a security layer using AES or TLS with Ascon for a messaging protocol called MQTT, which is a lightweight protocol suitable for very lightweight embedded systems such as Smart Badges (which use Bluetooth) or even for transmitting weather data from sensors, as in their paper. [9] The sensors and system detailed in their report would suffer when using more intensive security measures, but the risk of unauthenticated data over an insecure MQTT connection would be very high. [9] Situations involving very lightweight devices or sensors connected to other devices are a critical target for Ascon.

Given that analytics firm IoT-Analytics reports 18.5 billion IoT devices in use as of 2024, and that of those devices, 32% of them rely on Wi-Fi, and 24% on Bluetooth, [13] the importance of a lightweight, secure cryptographic family like Ascon is becoming greater by the year.

II. METHODOLOGY

A. Test Platforms and Hardware Specifications

A variety of different platforms were tested, ranging from a low-end ARM processor to a high-performance server. Representing constrained embedded devices, sensor nodes, and legacy IoT hardware, the Raspberry Pi Zero W is a legacy ARM platform with a mere single-core processor. Moving up to high-end ARM, the Raspberry Pi 5 represents modern high-performance ARM with hardware cryptography acceleration, typical of edge-computing devices and high-end IoT gateways. The StarFive VisionFive2 is a RISC-V platform representing emerging open-source ISA with growing embedded and IoT ecosystems. We also tested two x86-64 platforms. The first being a Qotom Mini PC, representing entry-level x86 embedded and industrial systems. The other x86 platform tested was an Intel Xeon server, establishing a performance ceiling for comparison, representing server-side cryptographic processing.

All platforms tested ran Linux-based operating systems, including Raspberry Pi OS, Debian, and Ubuntu derivatives to maintain consistency in benchmarking environments while acknowledging that bare-metal deployments would perform differently.

B. ASCON Implementations Tested

We evaluated multiple Ascon-128 AEAD implementations from the official Ascon GitHub repository to understand the impact of different optimization strategies. Beginning with a reference implementation (ref), this strategy provides portable C code without platform-specific optimizations. This reference implementation also acts as a baseline for comparing the other optimization strategies, and was compiled with -O3 optimization across all platforms.

We also ran optimizations based on 32 and 64-bit architectures. The 32-bit optimization (opt32) uses compiler-friendly patterns for instruction-level parallelism, which is particularly relevant for ARMv6 and 32-bit embedded systems. The 64-bit optimization (opt64) leverages 64-bit operations in Ascon's permutation layer, and was expected to perform best on ARMv8, RISC-V, and x86-64 systems.

There were also a variety of platform-specific implementations included in these tests, including *bi32_armv6*, a bit-interleaved 32-bit implementation for ARMv6, *armv6_lowsize*, a code-size optimized variant for ARMv8, and *NEON*, an ARM NEON SIMD implementation for ARMv8.

TABLE II-1 DEVICE INFO

Device	Efficiency (clock cycles / byte)	Data Throughput (MB/s)
Raspberry Pi 5	0.06	823
Raspberry Pi Zero	15.62	75
VisionFive2	7.08	144.68
Qotom	5.03	528.44
Xeon	0.85	1173.75

Some assembly implementations were included as well: *asm_rv32i*, RISC-V assembly using base integer instructions, *asm_rv32b*, RISC-V assembly with bit manipulation extension, and *asm_bi32_rv32b*, a bit-interleaves RISC-V assembly.

This comprehensive implementation matrix allowed us to identify optimal configurations per platform and assess whether hand-optimized assembly SIMD code provided meaningful advantages over compiler-optimized C code.

C. Benchmarking Framework

We developed a purpose-built C code benchmarking harness that extends the NIST LWC Standardization Project methodology [14] for single-board computer environments. Cycle-accurate measurements are a key feature of this code, including a generic timer using system registers for the Pi 5, software estimation using CPU frequency scaling data for the Pi Zero W, cycle counter CSR or time-based estimation for the VisionFive2, and time-stamp counters for the x86 platforms. High resolution timing is also a key feature, allowing for nanosecond precision wall clock measurements. This feature also integrates independent verification of cycle-based measurements and ensures representative steady-state performance.

Different types of test parameters were used to create a more customized benchmark for our application. Message sizes ranging from 16 to 8192 bytes were tested to measure throughput. Data associated with these messages was kept to 16 bytes for consistency, allowing us to measure the number of cycles per byte each platform had to execute. 1000 measurements per configuration were iterated, adjusted for message size, and encryption and decryption were measured independently to get a more accurate picture of the performance and efficiency of Ascon on each platform.

For the 1000+ timing samples collected in each configuration, we calculated the median, mean, standard deviation, and any outliers to assess distribution characterization and performance consistency. Results were exported in CSV format with embedded platform metadata for reproducibility and cross-platform comparison.

D. Experimental Setup

We employed platform-specific compiler optimization flags to represent realistic deployment scenarios. All platforms used GCC version 11.X or later, ensuring consistent optimization capabilities while respecting architecture-specific instruction sets.

To ensure a standardized environment and minimize measurement variance, a variety of practices and tools were implemented. A CPU governance configuration was developed, allowing us to define a “performance mode” on each platform. This configuration also allowed us to disable dynamic frequency scaling if we so choose, enabling stable CPU frequency throughout benchmarking. We also implemented system load minimization: disabling unnecessary services, choosing single-user mode where possible, disabling all network interfaces with the exception of remote access, and ensuring that there were no concurrent workloads during measurement. Thermal management was also imperative: all platforms were monitored for thermal throttling. Benchmarks were aborted and restarted if

```
# Raspberry Pi 5:
gcc -O3 -march=armv8.2-a+fp16+rcpc+dotprod+crypto \
    -mtune=cortex-a76 -flto -fomit-frame-pointer
# Raspberry Pi Zero W:
gcc -O3 -march=armv6 -mfpu=vfp -mfloat-abi=hard \
    -flto -fomit-frame-pointer
# VisionFive2:
gcc -O3 -march=rv64gc -mtune=sifive-7-series \
    -flto -fomit-frame-pointer
# x86-64 Platforms:
gcc -O3 -march=native -mtune=native \
    -flto -fomit-frame-pointer
```

CODE BLOCK 1: COMPILER OPTIMIZATION FLAGS

throttling was detected during measurement, and the use of passive cooling reduced the risk of performance variability caused by active cooling.

Each platform configuration was tested across 3-5 separate benchmark runs and their results were verified for consistency, accepting less than 5% variance between runs. We selected the median run of each platform configuration for reporting, investigating outlier runs for system-level interference. Results were aggregated using a custom Python script, which parsed CSV outputs from all platforms. This script also normalized metrics for cross-platform comparison, generated throughput scaling curves, and computed efficiency metrics. The result of this script produced publication-ready visualizations and graphs. This rigorous methodology ensures that observed performance differences reflect genuine architectural characteristics rather than measurement artifacts or environmental variations.

III. RESULTS

A. Cross-Platform Performance Overview

By running tests, it was found that architecture has a greater impact on processing speed than implementation choices since despite different hardware acceleration and optimization implementations, the throughput speed remained the same. As such, the following table only references the highest speed obtained by each processor and device, regardless of implementation of choice.

Platform-Specific Analysis

For the Raspberry Pi 5, ASCON was benchmarked using 32-bit ARM instructions, 64-bit ARM instructions, and NEON SIMD instructions. However, the optimization approach that produced the best results was NEON; and for the purposes of obtaining data, NEON’s results reflected the benchmarking speed of the Raspberry Pi 5. NEON’s implementation resulted in a 0.06 clock cycle per byte (which is approximately 823 MB/s throughput for a 2.4 GHz clock cycle). However, this does not mean that the 32-bit and 64-bit ARM instructions were slow. Minimal variance existed through each optimization method, which means that the Raspberry Pi 5, in general, is very efficient in running ASCON. One downside of having these instructions built in and available for use on the ARM processors was that the performance ceiling was apparent and that the fastest speed of any one hardware acceleration implementation is the fastest

that the processor will go. This is not to say that the speed measured was not efficient or satisfactory, but that improvement not possible.

Unlike the Raspberry Pi 5 and other newer generation Raspberry Pi's, the Raspberry Pi Zero does not have NEON for hardware acceleration. As such, the 32-bit and 64-bit ARM instructions were used to optimize the benchmarking process. Despite which ARM instructions were utilized, consistent measurements of approximately 75 MB/s were obtained, which is approximately 15.62 clock cycles per byte. For context, this is about 260 times slower than the Raspberry Pi 5. In this case specifically, the results show that as the Raspberry Pi developed, the instruction of modern optimizations and faster processors greatly improved the throughput of processes throughout the generations. However, it is important to note that ASCON was still able to operate at 75 MB/s on the Raspberry Pi Zero, which is now considered extremely limited and obsolete. As such, it goes to highlight that ASCON can still work well on hardware that is extremely limited.

Another device that the ASCON benchmark was used on was the VisionFive2. The VisionFive2 was able to obtain 144.68 MB/s across all hardware acceleration implementations, which is approximately 7.08 clock cycles per byte. One notable implementation was the assembly implementation of the benchmarking tool. It was found that assembly optimizations showed no improvement over C, which is interesting because assembly is supposed to run faster than C, since C is a high-level language and assembly is just machine code. Another critical finding was that the RISC-V ecosystem contains a maturity gap. Without proper tools, compilers, and optimization tools, RISC-V seemed to always be behind ARM, which allows for future improvement as these shortcomings are remedied in the future.

Lastly, two x86-64 platforms were implemented, one being the Qotom and the other being Xeon. The Qotom managed a throughput of 528.44 MB/s, while the Xeon managed a throughput of 1173.75 MB/s, which is the fastest speed measured in this test. One thing to note was that the speed on the Xeon equates to only 0.85 cycles/byte. This was possible because the Raspberry Pi 5 operates at 2.4 GHz, while the Xeon may operate between 3.5 to 4 GHz. Because the Xeon used much more clock cycles per byte than the Raspberry Pi 5 and produced a much higher speed than the Raspberry Pi 5, further optimization can probably occur and may bring the speed higher.

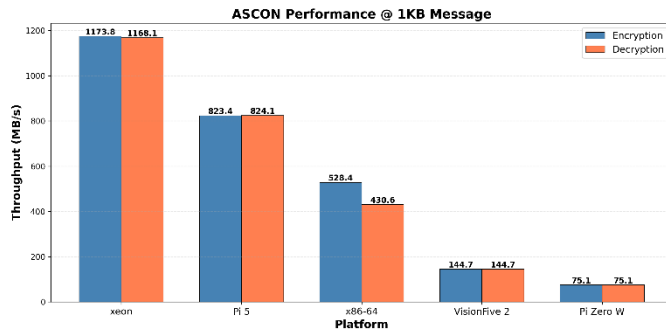


FIGURE 2 ASCON PERFORMANCE

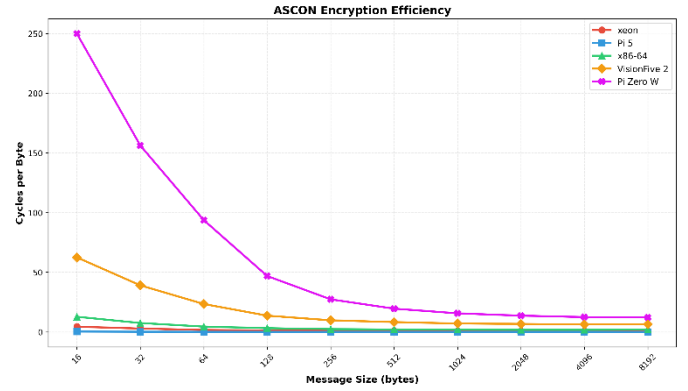


FIGURE 1 ASCON ENCRYPTION EFFICIENCY

B. Scaling Behavior Analysis

To test how each device would perform at different scales, the benchmark was run for different message sizes - 16 bytes, 32 bytes, 64 bytes, 128 bytes, 256 bytes, 512 bytes, 1024 bytes, 2048 bytes, 4096 bytes, and 8192 bytes.

From the data, it can be reasonably inferred that throughput and efficiency decrease as the message size increases. However, the extent of that decrease seemed to decrease as the message size increased. This is because of the fixed costs of ASCON encryption. The more bytes there are, the more this upfront and fixed cost decreases per byte. This is why small messages appeared to have higher overhead.

One notable analysis is convergence point analysis on each of the devices. For the Raspberry Pi Zero, it seemed to plateau at around 15-20 cycles/byte. For the VisionFive2, the curve flattened at around 10-15 cycles/byte. For Qotom, the curve flattened quickly and settled at around 3-5 cycles/byte. As for the Raspberry Pi 5 and the Xeon, the curves seemed to have flattened almost immediately at around 0.5 cycles/byte and 2-3 cycles/byte, respectively.

C. Implementation Comparison

Platform-specific optimizations, which in the case of the Raspberry Pi 5 are the 32-bit, 64-bit, and NEON ARM instructions, can improve the throughput and efficiency of a processor. For NEON SIMD on the Raspberry Pi 5 specifically, an improvement in performance was noted, but only slightly due to the hardware ceiling on the processor. Regardless, implementation does play a role in throughput and efficiency improvement for the Raspberry Pi 5. However, implementation of choice does not matter in certain cases, which is true for the VisionFive2. Whether the implementation occurred in C or Assembly, which would mean better efficiency and throughput for Assembly due to it being a machine language, the performance did not change. This can be attributed to the lack of modern tools on the VisionFive2.

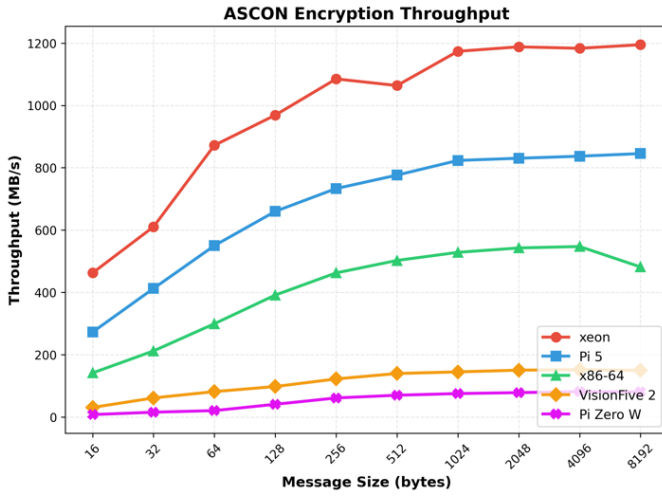


FIGURE 4 ASCON ENCRYPTION THROUGHPUT

DISCUSSION

D. Architectural Impact on Cryptographic Performance

As ARM processors evolved, each generation introduced features on their crypto engines that allowed higher throughput speeds of cryptographic algorithms in addition to increasing clock speed, which would also inherently increase throughput. This is seen on the Raspberry Pi 5 being 11 times faster than the Raspberry Pi Zero. One added feature present in modern ARM processors is NEON SIMD. This allowed for multithreading and better performance in general. However, due to the hardware ceiling present in these modern ARM processors, NEON only provided a slight performance advantage over regular scalar code (implemented through the 32-bit and 64-bit ARM instructions). Regardless, it is impressive that modern ARM processors, specifically those v8.2+, are competitive with x86-64 processors.

For the VisionFive2, which represented RISC-V processors, it was surprising that the 64-bit quad-code processor on the VisionFive2 underperformed. However, it makes sense, given that hardware acceleration and optimization tools have not fully developed for these processors. When RISC-V was released, the hope was that the custom, open-source package would be good and allow these types of processors to flourish. However, unlike ARM which is a commercial product and relies on technological advancements to make a marketable product, the open-source package has no pressure to evolve and will continue to move at the pace the community decides. As such, RISC-V will take some time to catch up to ARM in embedded cryptography.

In the test, it was found that Xeon achieved the absolute best throughput at 1173.75 MB/s. However, the high throughput obtained by Xeon can be easily attributed to the higher clock speed, since the Raspberry Pi 5 obtained 823 MB/s with a higher efficiency of 0.06 clock cycles / byte. Additionally, x86-64 processors have been around since the early development of the modern computers, which allows for those processors to have the edge in terms of technology and hardware acceleration implementations over ARM processors. However, some advantages that other processors have over x86-64 processors is

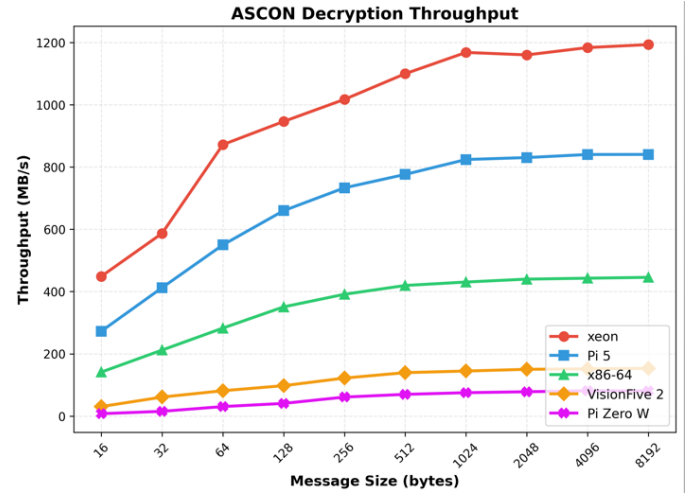


FIGURE 3 ASCON DECRYPTION THROUGHPUT

that the Xeon is expensive compared to other products. In addition, Xeon uses much more power compared to other lightweight devices. The specific tested device was a SoC on a server motherboard, a situation where power is at less of a premium. As such, although Xeon produced the highest throughput of all the devices used, it is seemingly unsuitable for inexpensive embedded systems. However, it is a good example of the performance to expect on the server side when using Ascon with a number of lightweight nodes. Intel specifies the Xeon D as a candidate for edge routing, wireless access, and other devices where there must be a balance of speed and power use, versus a very low power device like the Celeron in the Qotom mini PC or Raspberry Pi 5. [21]

E. Implications for IoT/Embedded Systems

For IoT and embedded systems, the Raspberry Pi 5 is good for edge gateways requiring high throughput, since it was competitive with Xeon in this test without the drawbacks of using an actual Xeon. For sensor nodes with periodic transmission, the Raspberry Pi Zero can work with ASCON, since despite producing the lowest throughput, the efficiency of the device was still acceptable. Additionally, 75 MB/s on a single-code ARM-v6 at 1 GHz exceeds many sensor requirements. Thus, in specific uses, the Raspberry Pi Zero can be used in IoT/Embedded Systems. As for RISC-V processors, specifically the VisionFive2, these processors are not ready for performance-critical cryptographic algorithms due to the immaturity of its ecosystem.

With ASCON having such high efficiency on modern processors (0.06 clock cycles/byte on the Raspberry Pi 5 and 0.85 on the Xeon), ASCON is perfect for lightweight devices. Additionally, ASCON's consistent performance across implementation variants means that the algorithm is reliable over multiple devices.

F. Future-Proofing Considerations

RISC-V is still developing its ecosystem, which means that this test, though it is accurate now, could yield different results in a few years. The potential of RISC-V crypto extensions is great and is expected to rival ARM in the future. As such, these test results are not final. However, now, ARM processors have

the advantage over the different processors, even the x86-64 processors, due to its lightweight and performance advantages. If the Xeon were to be implemented on embedded systems, then the embedded system would have to pay more in terms of monetary cost, power, and space. However, the tradeoff for just 300 more MB/s at a lower efficiency does not make sense. This means that ASCON will likely continue to run on ARM processors in the future because of the efficiency of ASCON on these processors.

IV. LIMITATIONS AND FUTURE WORK

A. Current Study Limitations

Some limitations that arose in this study is that there was only one representative in each architecture class. Specifically, if more devices used RISC-V processors, it could be more easily observed if there were more issues present than just ecosystem maturity. If there were more devices in the test that used a specific architecture class, it would be easier and more concrete to decide which type of processor ASCON would run better on. Additionally, Linux overhead used some resources, thus skewing the results slightly. If ASCON were able to be observed in action, rather than on a benchmark with Linux overhead, the efficiency of ASCON would be much more accurate. However, getting this level of efficiency with some Linux overhead says a lot of the efficiency of ASCON in general. Lastly, power consumption for ASCON on these processors was not measured. This measurement would give another reason as to why a certain processor would be better to run ASCON on in the future. If a future test somehow overcomes these limitations, a more concrete and comprehensive test on which processor would be best to run ASCON on an embedded system would be possible.

B. Future Research

In the future, an energy consumption measurement would also help to show which device ASCON would work better on, since that is one of the considerations when choosing what device to run ASCON on. This would also help to show if applying hardware acceleration on ASCON in embedded systems would be worth the extra power consumption for a higher throughput. Energy consumption measurements should also include thermal analysis, since the efficiency of the circuit should also be taken into consideration when determining which device ASCON works better on. Additionally, the implementation of bare-metal benchmarks would better show the efficiency of ASCON on these devices since resources used by the OS would be eliminated. This will show the actual, minimal resources required to implement ASCON. Furthermore, one area that was disappointingly lacking was the immaturity of the RISC-V ecosystem. In a future test, once the RISC-V ecosystem is more evolved and more hardware acceleration and optimization tools for cryptographic algorithms are available, the test should be redone. Additionally, it would be interesting to see how ASCON is used in real-life applications and how the throughput and efficiency changes when in these situations.

V. CONCLUSION

Through this test, it was found that the device with the fastest throughput was Xeon, which belonged to the x86-64, while the slowest throughput was the Raspberry Pi Zero. It makes sense

that the x86-64 devices would have a higher throughput because of the clock speed of those devices. It also makes sense that the Raspberry Pi 5 would produce a higher-end throughput because of the later release date of the processor compared to the other parts. On the other end, the Raspberry Pi Zero producing the slowest throughput makes sense because it contains what is considered a legacy ARM processor. What is surprising is that the Raspberry Pi 5, which represents modern ARM processors, can be competitive with x86-64 processors. Since x86-64 processors are used on computers, it can be deduced that these processors would greatly outperform ARM processors. However, the Raspberry Pi 5 produced the second fastest throughput. It would be interesting to see how RISC-V matures in the future and how its maturity would affect the throughput of ASCON through RISC-V processors. Additionally, by implementing ASCON on a RISC-V processor, the test also showed that hardware cryptography extensions matter more than hand-optimized code, since running the benchmark in C and assembly did not change the throughput.

From the data obtained through this test, the Raspberry Pi 5 can be recommended for high-throughput edge devices, while the Raspberry Pi Zero should be constrained for IoT sensors due to its lesser efficiency in implementing ASCON. Furthermore, RISC-V can be adopted later if the ecosystem matures and tools/hardware are developed to increase the efficiency of such algorithms.

Regardless of device and throughput, ASCON was found to be an efficient lightweight cryptographic algorithm due to its maximum efficiency being 0.06 clock cycles/byte and its least efficiency being 15.62 clock cycles/byte. One thing to note is that the 15.62 clock cycles/byte was obtained on what represented a legacy ARM processor. As such, on more modern processors that would be used, it can be inferred that greater efficiency would be obtained. This is an important finding because in the millions of IoT devices that use such processors, if the seemingly worst efficiency is still good enough, then it shows that ASCON is a good algorithm for security in lightweight cryptographic devices. As such, because the algorithm was tested on lightweight portable devices, it can be inferred that the algorithm is, indeed, much more suitable for such devices than AES.

VI. REFERENCES

- [1] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  fer, "Ascon v1.2: Lightweight Authenticated Encryption and Hashing," *Journal of Cryptology*, vol. 34, no. 3, pp. 32-42, Jun. 2021, doi: <https://doi.org/10.1007/s00145-021-09398-9>.
- [2] A. Bozhko, Ed., "Properties of AEAD Algorithms," *Ietf.org*, Feb. 29, 2024. <https://www.ietf.org/archive/id/draft-irtf-cfrg-aead-properties-04.html#RFC5116> (accessed Dec. 05, 2025).
- [3] M. S. Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, NIST SP 800-232 Ascon-Based Lightweight Cryptography Standards for Constrained Devices. National Institute of Standards and Technology (NIST), 2025. doi: <https://doi.org/10.6028/nist.sp.800-232>.
- [4] Internet Society, "What is TLS & How Does it Work? | ISOC Internet Society," Internet Society, 2025. <https://www.internetsociety.org/deploy360/tls/basics>
- [5] J. Zhou, W. Fu, W. Hu, Z. Sun, T. He, and Z. Zhang, "Challenges and Advances in Analyzing TLS 1.3-Encrypted Traffic: A Comprehensive Survey," *Electronics*, vol. 13, no. 20, pp. 1-9, Oct. 2024, doi: <https://doi.org/10.3390/electronics13204000>.

- [6] "Google Transparency Report," Google.com, 2025. <https://transparencyreport.google.com/https/overview> (accessed Dec. 06, 2025).
- [7] J. Harris, "The Eclipse Foundation Unveils 2024 IoT & Embedded Developer Survey Results," Eclipse News, Eclipse in the News, Eclipse Announcement, Dec. 03, 2024. <https://newsroom.eclipse.org/news/announcements/eclipse-foundation-unveils-2024-iot-embedded-developer-survey-results> (accessed Dec. 07, 2025).
- [8] C. Boutin, "NIST Selects 'Lightweight Cryptography' Algorithms to Protect Small Devices," NIST, Feb. 2023. Available: <https://www.nist.gov/news-events/news/2023/02/nist-selects-lightweight-cryptography-algorithms-protect-small-devices>
- [9] J. Avery et al., "Analysis of Practical Application of Lightweight Cryptographic Algorithm ASCON," 2022. Available: <https://csrc.nist.gov/csrc/media/Events/2022/lightweight-cryptography-workshop-2022/documents/papers/analysis-of-practical-application-of-lwc-cryptographic-algorithm-ascon.pdf>
- [10] Bluetooth SIG, Inc., "Specification of the Bluetooth® System, Part E: Low Energy Link Layer Security," Bluetooth.com, 2025. <https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/Core-54/out/en/low-energy-controller/low-energy-link-layer-security.html> (accessed Dec. 07, 2025).
- [11] Cisco Systems, Inc., "Wireless Fundamentals: Encryption and Authentication," Cisco Meraki Documentation, Oct. 05, 2020. https://documentation.meraki.com/Wireless/Design_and_Configure/Architecture_and_Best_Practices/Wireless_Fundamentals%3A_Encryption_and_Authentication (accessed Dec. 07, 2025).
- [12] Wi-Fi Alliance, "Wi-Fi Protected Access ® : Security for Wi-Fi ® networks," Apr. 2025. Available: https://www.wi-fi.org/system/files/Security_Highlights_202504.pdf
- [13] S. Sinha, "State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally," IoT Analytics, Oct. 28, 2025. <https://iot-analytics.com/number-connected-iot-devices/>
- [14] NIST, "Lightweight Cryptography Benchmarking," GitHub repository, 2023. [Online]. Available: <https://github.com/usnistgov/Lightweight-Cryptography-Benchmarking>
- [15] P. Schwabe, B. Westerbaan, and B. B. Yilmaz, "X25519 and Ed25519 for ARM Cortex-M4 and Cortex-A," IACR Cryptology ePrint Archive, 2023. [Online]. Available: <https://eprint.iacr.org/2023/891>
- [16] ARM Limited, "ARM Cortex-A76 Software Optimization Guide," Version 1.0, 2022. Available: <https://developer.arm.com/documentation/swog309953/latest/>
- [17] B. Marshall, G. R. Newell, D. Page, M.-J. O. Saarinen, and C. Wolf, "The Design of Scalar AES Instruction Set Extensions for RISC-V," IACR Transactions on Cryptographic Hardware and Embedded Systems, vol. 2021, no. 1, pp. 109-136, 2021, doi: <https://doi.org/10.46586/tches.v2021.i1.109-136>
- [18] RISC-V International, "RISC-V Cryptography Extensions Volume I: Scalar & Entropy Source Instructions," Version 1.0.1, 2023. Available: <https://github.com/riscv/riscv-crypto/releases/download/v1.0.1-scalar/riscv-crypto-spec-scalar-v1.0.1.pdf>
- [19] A. Langley, "AES-NI and PCLMULQDQ Performance," Imperial Violet (blog), Oct. 2010. Available: <https://www.imperialviolet.org/2010/10/20/aesgcm.html>
- [20] M. Hamburg, "Fast and Compact Elliptic-Curve Cryptography," Cryptology ePrint Archive, Paper 2012/309, 2012. [Online]. Available: <https://eprint.iacr.org/2012/309>
- [21] "Intel® Xeon® Processor D-2100 Product Family Technical Overview," Intel, 2018. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-xeon-processor-d-2100-product-family-technical-overview.html> (accessed Dec. 08, 2025).